目次

入力と計算量 2
まず全探索 3
一気に 2 つ以上のことをしようとしない 4
操作をする問題 5
数え上げ問題 6
区間問題 7
ゲーム 8
制約を満たすものを数える 9
構築系問題 10
K 番目の数を求める 11
xor 12
マンハッタン距離(L-1 距離) 13
グラフ系 14
条件を満たす最大値 or 最小値を求める 15
偶奇性に注目する 16
最大マッチング 17
積の総和は分配法則で和の積にする 18
二次元座標を二部グラフで考えてみる 19
順序がある場合は有向グラフで考えてみる 20
凸関数の極値は三分探索 21
数列の番号に対する単調増加な性質はしゃくとり法 22
等比数列や〇進数は割った余りで考える 23
括弧列は山の上り下り(折れ線)として考える 24

入力と計算量

・N=8 の場合 O(N!)
・N=10~20 の場合 O(2^N) (bit 全探索)
　　N が大きくなければ O($N^2 2^N$) (bitDP や包除の原理)
・N=30~40 の場合 O($N2^{N/2}$) (O(2^N)の半分全列挙)
・N=50 の場合 O(N^4)
・N=300~500 の場合 O(N^3) (区間 DP など)
・N=1000 の場合 O(N^2log(N)) (O(N^4)の半分全列挙、一回 O(Nlog(N))の二分探索)
・N=3000 の場合 O(N^3)
・N=10^5 の場合 O(Nlog(N))など
・N>10^{18} の場合文字列で受け取り桁ごとに考える
・mod が小さい場合余りの全探索
・意味深な定数が存在する場合
　その数の性質が良いことを利用する（素数，約数に特殊な数がある，など）
　その数が小さいことを利用する（メモリの確保が容易で DP が使える，など）

まず全探索

10^7 回のループなら余裕をもって回せる
難しい問題でも「計算量を落として最後は全探索」というパターンは非常に多い
高度なアルゴリズムを用いるよりも愚直な全探索

・ループによる全探索
・ビット全探索（2^n 通り）
・順列(n!)の全探索
・DFS による全探索

一気に 2 つ以上のことをしようとしない

分解して考える（シンプルな場合を考える）
足し算を分ける
操作によって得られる結果を分ける
2 次元の問題は、1 次元に分解して独立に考える
入力が「整数の範囲」なら、「正の数だけの場合」「負の数だけの場合」「0 だけの場合」
「1, 2, 3 しか登場しない」なら、「2 と 3 だけの場合」

変数がいくつかある時は、一方を固定して考える

対称性を活かす
求めたいものと、それと対になる部分（対称な部分）の値が等しいとき、その 2 つの和の方が計算しやすい場合がある

x 座標と y 座標の対称性から 1 次元の問題に帰着

操作をする問題

ある操作を「K 回以内行う」とか「好きなだけ行う」とか指定されている問題は、操作ごとに不変なものを考えてみる
操作をしても偶奇性が変化しない場合や、逆に変化する場合に注目する
操作の順番が決められている問題の時、結局どの順番で操作を行ったとしても結果が変わらない場合がある
逆順に操作をすることを考えると分かりやすくなる場合がある
操作をして変換したものを、また操作することによって、元に戻すことが可能かどうかを見る(同値類に分類)

数え上げ問題

全探索で探索する状態をいくつかにまとめて同一視することで、動的計画法を使える
いくつかの数列があり、平均が K となる組み合わせを求めるときは、
・「合計値が K × 個数」となる組み合わせ
・「元の値をすべて-K した数列で、合計値が 0」となる組み合わせ
というように変更すると考えやすくなることがある

区間問題

・左右両方から累積和を前処理して保存しておくと、後の計算を効率よくできる場合があります。一つだけ除いたものを考える場合なども有効
・区間を足す時 imos 法や、区間和対応の BIT などで解ける
・区間を削除・圧縮・合成する時区間 DP
・公差 d の等差数列を区間に加算する場合は、加算した値に差がどれだけ変化するかに注目してみると、そのほとんどが公差 d だけ変化するのに注目すると分かりやすくなる

区間を反転させる問題
以下のことに注意
・同じ操作を 2 回やると元に戻る
・操作の順番を入れ替えても同じ
・[l1, r1]と[l2, r2] の反転は、[l1, r2]と[l2,r1] の反転と同じ（区間の左端はどのように反転させても左端、右端はどのように反転させても右端）
・反転区間の端点に注目すると greedy(貪欲)に決まることがある

ゲーム

不偏ゲームの場合は、Grundy 数で先手後手のどちらが必勝かを判定する
ゲームが終了した時からさかのぼって考えると最適な行動が分かる
いくつか手段があるときは、同じ手段を取り続けるのが最善の時がある

一方が最大化・他方が最小化
まず基本となるのは最善となる戦略を先手・後手ともに見つけることです。
後手は「先手がどう動くか」が分からないと方針が立たないことが多いので、先手の行動を場合分け
最大化側は x 以上・最小化側は x 以下を達成できるか

制約を満たすものを数える

直接考えるのが難しい時は余事象

構築系問題

「構築できない場合」が存在しないことがある
上限ギリギリでの構築法が全てに当てはまることがある
「2N 回以内に終了」なら N 回以内に特殊な状況にもっていく

K 番目の数を求める

要素の値が小さい時 BIT などで K 番目の数を求める
二分探索で求める

xor

xor は繰り上がりのない足し算
a+b=a⊕b+2*(a&b)が成立
xor は桁ごとに考える
同じ数を xor すると元に戻る

マンハッタン距離(L-1 距離)

座標を 45 度回転させることにより、あるマンハッタン距離 d で移動可能な範囲は正方形の形になります。
元の座標が（x, y）であれば、新しい座標で（X,Y)=(x+y, x-y）などと変換してみます。すると、[X-d, X+d] かつ [Y-d, Y+d] の範囲が元々の座標で移動可能だった範囲です
差の最小化は中央値を用いる(点との距離の和なので三角不等式より間にあれば最小)

グラフ系

以下の代表的なグラフで考えてみる

・パスグラフ（直線上に繋げたグラフ）

・スターグラフ（一つの頂点から、他のすべての頂点に 1 つずつ辺が出ているグラフ）

・完全グラフ（任意の 2 頂点間に辺があるグラフ）

・完全二部グラフ（二部グラフのうち、可能な限り辺をつないだもの）

・パスグラフに少し辺を足したグラフ

木は二部グラフであることを利用する

木の直径を考えてみる

条件を満たす最大値 or 最小値を求める

条件を満たすように最大化/最小化は二分探索。例えば以下

・平均の最大化・最小化

・最大値の最大化

・最小値の最大化

選択肢が少ない方から見て貪欲

偶奇性に注目する

abs(x-y) などの計算では、偶奇性（パリティ）などに注目すると上手くいくことがある

・abs(x-y) が奇数：x と y の偶奇が異なる

・abs(x-y) が偶数：x と y の偶奇が同じ

$|x-y| \equiv x \oplus y \equiv x+y \pmod 2$

xor で偶奇を見る

奇数を偶奇性でみる

木においては、木を二部グラフとしてみると、偶奇性が分かりやすくなる

最大マッチング

フローを利用した二部グラフの最大マッチング(有名なアルゴリズム)
二部グラフではなかったり、エッジの数が膨大になりそうな時は、貪欲にマッチングが取れることがあります。その場合は、「選択肢が少ない方から見て貪欲」に取るのが最適なことが多い

積の総和は分配法則で和の積にする

和の積の形に変形できれば計算量が落とせることがある

二次元座標を二部グラフで考えてみる

二次元の平面上にあるいくつかの点を、以下のような二部グラフで表現してみるとうまくいくことがあります。

・二部グラフの左側：x 座標
・二部グラフの右側：y 座標
・二部グラフの辺（x,y）：点（x,y）に対応

順序がある場合は有向グラフで考えてみる

順序関係が存在するような問題設定の場合は、有向グラフの問題に帰着できる場合があります。
特に半順序集合の場合は、閉路のない有向グラフ(DAG)によって表現することができ、これはトポロジカルソートすることができます。

凸関数の極値は三分探索

凸関数の極値を求める問題では、三分探索や黄金分割法などを用いて求めることができます。

数列の番号に対する単調増加な性質はしゃくとり法

l を固定したときに、数列の l 番目から r 番目が条件を満たすような最大の r を f(l) と表せる時を考えます。この f が単調増加のときにはしゃくとり法を用いることができます。

等比数列や〇進数は割った余りで考える

等比数列や〇進数のような問題は、割った余りに注目してみると良い

括弧列は山の上り下り(折れ線)として考える

'(' や ')' が並んだ文字列を考える問題では、括弧がうまく閉じるかどうかなどを考えることがあります。このような問題では、

・'(' の時は上に 1 上る

・')' の時は下に 1 下る

というようにすると、考察しやすくなります。

この時、括弧がうまく閉じる条件は

・はじめと終わりは 0

・途中で 0 未満になることはない

と言い換えることができます。